Short-Term Demand Forecasting for CitiBike NYC: A Multi-Model Benchmark

Skye Xi
rx2285@nyu.edu

Data Bootcamp
Professor: Jacob Frias Koehler

1 Introduction

Accurate short-term demand forecasting is a core operational challenge for shared mobility systems. Operators must balance bike availability against redistribution costs, and a reliable 72-hour ahead forecast at the station level can directly inform rebalancing logistics. CitiBike, New York City's docked bike-share system, sees sharp spatial heterogeneity: high-traffic Midtown Manhattan stations can experience 10× more demand than quieter residential stations. Demand also exhibits strong temporal structure driven by commuting patterns, weekend leisure rides, and weather.

This project benchmarks six forecasting models on hourly CitiBike demand for 100 stations across NYC using data from January 2024 through December 2025. The models range from a non-parametric historical average to a Spatio-Temporal Graph Neural Network (STGNN) that explicitly encodes the geographic proximity graph among stations. The key findings are: (1) all neural models reduce MAE by 35–39% over the historical average, with LSTM achieving the lowest overall error (−39.1%); (2) STGNN with asymmetric log-cosh loss achieves the lowest peak-hour error among all models, outperforming LSTM during morning and evening rush hours; (3) adding a weather branch provides no benefit at this data scale; and (4) an asymmetric loss function that penalizes under-prediction improves peak-hour coverage with no meaningful cost to off-peak accuracy.

Exploratory data analysis is available at `github.com/skyexry/citi-bike`. All modeling code and notebooks are available at `github.com/skyexry/urban-mobility-forecast`.

2 Data Description

Trip records. Raw trip-level data were downloaded from the CitiBike system data portal in CSV format and converted to Parquet for efficient columnar storage and fast I/O during training. Each record contains departure station ID, latitude, longitude, and start times-

tamp. Trips were aggregated into an hourly demand count per station, yielding a $(T \times N)$ matrix of shape $(16{,}808 \times 100)$ covering 700 days. *(Notebook: `01a_preprocess.ipynb`)*

Station selection. From roughly 2,000 active stations, the top 100 by total annual demand were retained. A graph-connectivity filter removed any station with zero edges under the proximity graph to prevent isolated nodes from degrading spatial learning. *(Notebook: `01b_station_selection.ipynb`)*

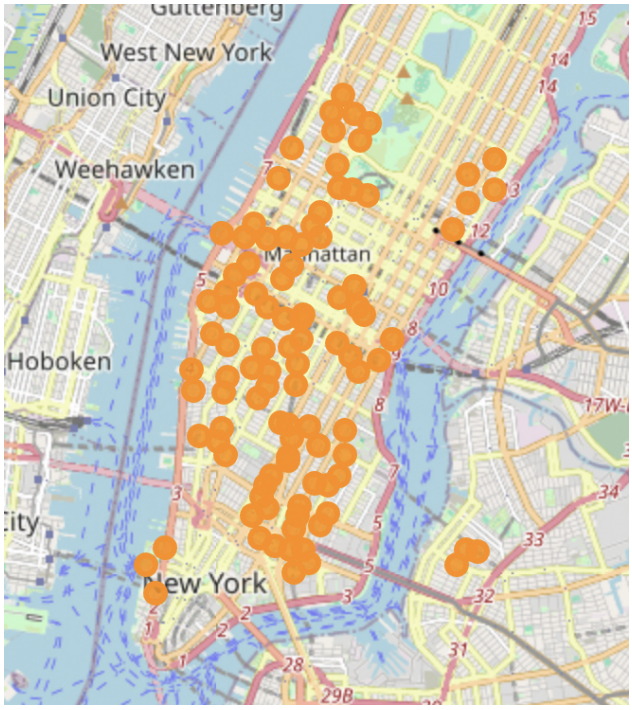

Figure 1: Geographic distribution of the 100 selected CitiBike stations across Manhattan and surrounding areas.

Feature engineering. Demand values were transformed with $\log(1 + x)$ to reduce skew from occasional extreme counts, then scaled to $[-1, 1]$ using a Min-Max scaler fit exclusively on training data to prevent leakage. Cyclic time encodings (sin/cos of hour-of-day, day-of-week, and day-of-year) produce a 6-dimensional time feature vector aligned with each hourly step. *(Notebook: `03_features_test.ipynb`)*

Weather. Hourly weather for NYC was fetched from the Open-Meteo archive API (no API key required). Nine features were used: temperature, dewpoint, relative humidity, surface pressure, wind U/V components, wind gust U/V, and raw precipitation. The first eight were z-score normalized; precipitation was kept in mm/hr. *(`preprocessing/features.py`: `fetch_weather_data`, `build_weather_features`, `normalize_weather_features`)*

Train/validation/test split. Data were split chronologically: 70% training, 15% validation, 15% test, to respect the temporal ordering and avoid future leakage. A sliding window of 168 hours input (one week) / 72 hours output was used for all neural models (100 stations, $\sim$11,500 training samples). *(Notebook: `11_train_eval.ipynb`)*

Graph construction. A station proximity graph was built using the Gaussian kernel on pairwise Euclidean distances between station coordinates:

$$w_{ij} = \exp\left(-\frac{d_{ij}^2}{\sigma^2}\right), \quad w_{ij} = 0 \text{ if } w_{ij} < \theta,$$

The bandwidth $\sigma^2 = 0.0005$ and sparsity threshold $\theta = 0.90$ were selected by grid search over graph connectivity statistics (edge count, average degree), targeting a balance between sufficient connectivity and over-smoothing from excessively dense graphs. This yields 656 directed edges among the 100 nodes. *(Notebook: 02_graph_tuning.ipynb)*

3 Models and Methods

All neural models were trained on a single NVIDIA GeForce RTX 5090 GPU using PyTorch and PyTorch Geometric. The Adam optimizer was used with initial learning rate 10^{-3}, ReduceLROnPlateau scheduling (factor 0.5, patience 5), and early stopping.

Training vs. evaluation loss. Each model is optimized on its designated training loss (specified per model below). Early stopping and learning-rate scheduling are governed by validation mean absolute error (MAE),

$$\text{MAE} = \frac{1}{|\mathcal{S}|} \sum_{(i,n,t)\in\mathcal{S}} |y_{n,t}^{(i)} - \hat{y}_{n,t}^{(i)}|,$$

where $\mathcal{S}$ indexes over samples i, stations n, and forecast steps t. MAE is model-agnostic and directly interpretable in original demand units. Final test-set performance is reported as MAE and root mean squared error (RMSE) after inverse-transforming predictions back to trips per station per hour.

Historical Average (HA). For each station and hour-of-day (0–23), the mean demand over the training set is predicted directly. No parameters are learned. This baseline captures daily periodicity but ignores week-to-week variation and spatial correlations.

LSTM. A two-layer LSTM with 64 hidden units and dropout 0.2 is applied independently to each station's univariate demand sequence. A linear head maps the final hidden state to the 72-step output. Trained with symmetric log-cosh loss. This model captures temporal autocorrelation but uses no cyclic time features and is blind to spatial relationships between stations.

TCN-only. A Temporal Convolutional Network with 4 layers of dilated causal convolutions (dilation rates 1, 2, 4, 8; 64 channels) runs two parallel branches per node: one on the univariate demand sequence and one on the 6-dimensional cyclic time features. The two branch outputs are concatenated and passed to a linear head producing the 72-step forecast. Trained with symmetric log-cosh loss. Like LSTM, this model has no spatial graph component.

STGNN. The full Spatio-Temporal Graph Neural Network combines three branches:

1. *Spatial branch*: Chebyshev spectral graph convolution (`ChebConv`, $K = 3$, hidden channels 32) propagates information along the proximity graph edges at each time step.
2. *Demand TCN*: 4-layer dilated convolutions (64 channels) encode the local temporal pattern in each node's demand history.
3. *Time TCN*: encodes the shared cyclic time features and broadcasts them to all nodes.

The three streams are fused and decoded by a Transformer decoder head (2 layers, cross-attention) that attends across the output horizon to produce the 72-step forecast. We train two variants differing only in their loss function. **STGNN (asym)** uses an asymmetric log-cosh loss:

$$\mathcal{L} = \frac{1}{|\mathcal{B}|} \sum_i w_i \cdot \log\cosh(y_i - \hat{y}_i), \quad w_i = \begin{cases} 2.0 & \text{if } y_i > \hat{y}_i \\ 1.0 & \text{otherwise,} \end{cases}$$

which penalizes under-prediction twice as much as over-prediction to improve peak-hour coverage. **STGNN (sym)** uses the standard symmetric log-cosh loss ($w_i \equiv 1$) as an ablation to isolate the effect of the asymmetric penalty. Early stopping patience is set to 20 for both variants.

STGNN + Weather. A fourth TCN branch ingests the 9-dimensional weather features and is fused with the other three streams before decoding. All other hyperparameters are identical to STGNN (asym).

4 Results and Interpretation

Table 1 reports MAE and RMSE on the held-out test set, in original demand units (trips per station per hour). Peak MAE is computed over morning (7–9 am) and evening (5–7 pm) rush hours, averaged across all 100 stations. Figure 3 additionally breaks down peak errors by hour of day for the top-30 highest-demand stations, where absolute errors are larger and operationally more consequential; those numbers are higher than Table 1 because they reflect the hardest-to-forecast stations only.

Table 1: Test set performance on 100 NYC CitiBike stations (72-hour forecast horizon). Peak/off-peak MAE averaged over all 100 stations during peak hours (7–9 am, 5–7 pm).

Model	MAE	RMSE	ΔMAE vs HA	ΔRMSE vs HA	Peak MAE	Off-peak MAE
Historical Average	13.18	21.20	—	—	—	—
LSTM	8.02	13.09	−39.1%	−38.3%	12.58	6.51
TCN-only	8.28	13.84	−37.2%	−34.7%	12.83	6.76
STGNN (sym)	8.20	13.69	−37.8%	−35.4%	12.36	6.81
STGNN (asym)	8.18	13.42	−37.9%	−36.7%	12.35	6.79
STGNN + Weather	8.49	14.07	−35.6%	−33.6%	13.28	6.89

Temporal models are strong baselines. LSTM achieves the lowest overall MAE (8.02, −39.1% vs HA), outperforming all graph-based variants on the aggregate metric. TCN-only underperforms LSTM (8.28 vs 8.02) despite comparable capacity. On a two-year dataset, LSTM's sequential inductive bias may generalize better than TCN's parallel dilated convolutions; alternatively, the two-branch concatenation of demand and time features in TCN-only may not integrate the two signals as effectively as LSTM's gated recurrence over the raw sequence.

Spatial structure improves peak-hour accuracy. While STGNN (asym) does not surpass LSTM in overall MAE, it achieves the lowest peak-hour MAE among all models (12.35 vs LSTM's 12.58, a 1.8% reduction), though the absolute gap of 0.23 trips/hr is small and should be interpreted cautiously. Geographic proximity is a weaker proxy for demand correlation than temporal history on this dataset—the sparse graph (656 edges, $\theta = 0.90$) and limited two-year training window constrain the spatial signal—yet the graph structure still provides a measurable advantage where it matters most: high-demand, high-correlation station clusters in central Manhattan.

Weather does not help. STGNN + Weather is the worst-performing neural model (MAE 8.49, −35.6% vs HA). The primary reason is that weather is a global signal: all 100 stations receive the same NYC weather reading, so the branch cannot learn station-level spatial differentiation regardless of data volume. A secondary factor is that two years of data may be insufficient to identify reliable global demand–weather relationships (e.g., rain suppressing system-wide demand) against the noise of other demand drivers.

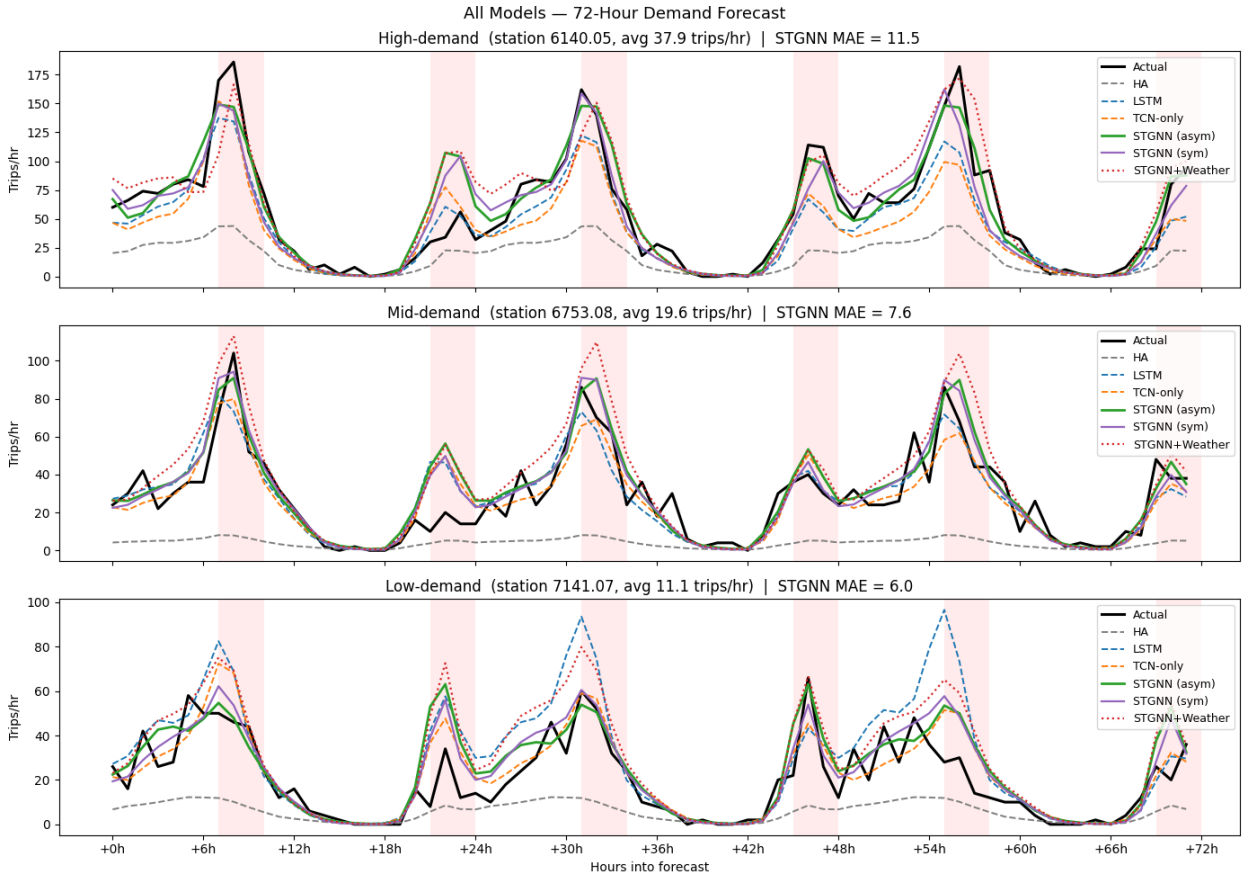

Figure 2: 72-hour forecasts for a representative high-, mid-, and low-demand station. Shaded bands mark peak hours (7–9 am, 5–7 pm). STGNN (asym) tracks demand peaks most closely at the high-demand station.

Asymmetric loss targets peak-hour coverage. Comparing STGNN (asym) to STGNN (sym) isolates the effect of the loss function. The asymmetric variant achieves lower peak MAE (12.35 vs 12.36) and lower overall RMSE (13.42 vs 13.69), confirming that penalizing under-prediction by a factor of 2 steers the model toward better coverage during high-demand hours with no meaningful cost to off-peak accuracy.

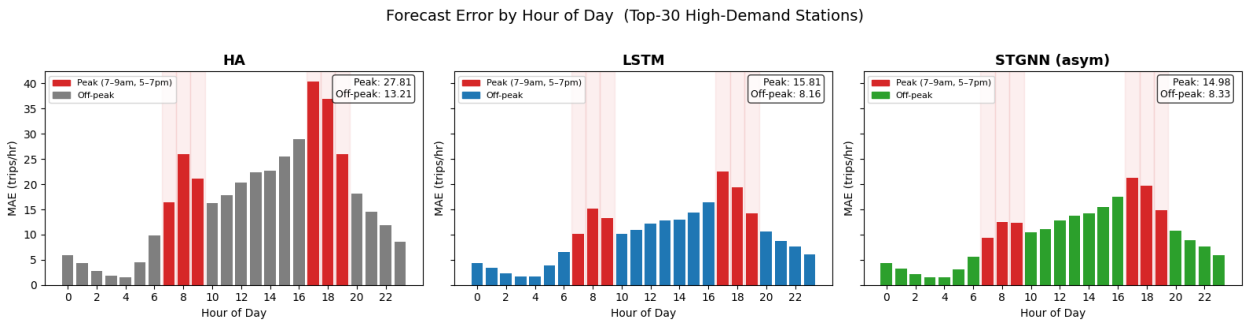

Figure 3: Forecast error by hour of day on the top-30 high-demand stations. Red bars indicate peak hours (7–9 am, 5–7 pm). STGNN (asym) achieves the lowest peak MAE (14.98) among the three models, outperforming LSTM (15.81) during morning and evening rush hours.

5 Conclusion and Next Steps

This project demonstrates that neural forecasting models achieve substantial improvements over the historical average baseline for CitiBike demand prediction, with all neural models reducing MAE by 35–39%. LSTM achieves the strongest aggregate performance (−39.1% MAE vs HA), while STGNN with asymmetric log-cosh loss achieves the lowest peak-hour error (12.35 vs LSTM's 12.58, −1.8% at peak), suggesting that architecture and loss-function choices can be tuned toward specific operational regimes, though the absolute peak gap (0.23 trips/hr) is small.

Several directions remain for future work. First, a *denser graph* with a lower sparsity threshold (tested but abandoned due to over-smoothing from uniform ChebConv averaging) could benefit from edge normalization or attention-weighted aggregation (e.g., GAT layers) to preserve spatial gradients. Second, *weather features* may become useful with multi-year training data that spans diverse seasonal patterns; the current two-year dataset provides insufficient weather variation to learn stable interaction terms. Third, *station metadata* (e.g., dock capacity, proximity to transit, neighborhood land use) could serve as static node features to help the model distinguish structurally different stations rather than relying solely on temporal history. Finally, extending the output horizon beyond 72 hours and evaluating probabilistic calibration (e.g., prediction intervals) would better support operational rebalancing decisions that require uncertainty quantification.